



Bahasa C++: Contoh Program Kecil

IF2210 – Semester II 2025/2026

Satrio Adi Rukmono

Counter: Prosedural vs. OO

Menunjukkan:

- Objek punya *tanggung jawab* dan *invariant* (*counter* tidak boleh negatif).
- Interface kecil, state tersembunyi.

```

// counter_procedural.cpp

// "Data structure" (hanya data)
struct Counter {
    int count;
};

// "API" prosedural (himpunan fungsi)
void counter_init(Counter* c) { c->count = 0; } // Kontrak implisit: count tidak boleh negatif.

void counter_inc(Counter* c) { ++c->count; }

bool counter_dec(Counter* c) {
    if (c->count == 0) { return false; } // cegah jadi negatif
    --c->count;
    return true;
}

int counter_value(const Counter* c) { return c->count; }

void cheat_make_negative(Counter* c) { // Contoh "pelanggaran kontrak"
    c->count = -123;                  // tidak ada mekanisme yang mencegah ini
}

```

```

// main.cpp

#include "counter_procedural.cpp"
#include <iostream>
#include <cstdio>

int main() {
    Counter c;
    counter_init(&c);

    counter_inc(&c); counter_inc(&c);

    std::cout << "value = " << counter_value(&c) << "\n";

    if (!counter_dec(&c)) { std::printf("Gagal decrement\n"); } // guard harus eksplisit

    std::cout << "value = " << counter_value(&c) << "\n";

    // Contoh: programmer bisa langsung memodifikasi data
    cheat_make_negative(&c);
    std::cout << "value (after cheating) = " << counter_value(&c) << "\n";

    return 0;
}

```

```
// counter.cpp

#include <stdexcept>

class Counter {
public:
    void increment() { ++count_; }

    void decrement() {
        if (count_ == 0) {
            throw std::runtime_error("Counter tidak boleh negatif");
        }
        --count_;
    }

    int value() const noexcept { return count_; }

private:
    int count_{0}; // invariant: count_ >= 0
};
```

```
// main.cpp

#include "counter.cpp"
#include <iostream>

int main() {
    Counter c;

    c.increment();
    c.increment();
    std::cout << "value = " << c.value() << "\n";

    c.decrement();
    std::cout << "value = " << c.value() << "\n";

    // c.decrement(); c.decrement(); // uncomment untuk lihat invariant dijaga
    return 0;
}
```

Invariant + Constructor: “BoundedStack”

Constructor memastikan objek lahir valid (invariant), bukan sekadar inisialisasi field.

```

// bounded_stack.cpp
#include <iostream>
#include <vector>
#include <stdexcept>

class BoundedStack {
public:
    BoundedStack(int capacity=128) : capacity_{capacity} {
        if (capacity_ <= 0) { throw std::invalid_argument("capacity harus > 0"); }
        data_.reserve(static_cast<std::size_t>(capacity_));
    }

    void push(int x) {
        if (size() >= capacity_) { throw std::runtime_error("Stack penuh"); }
        data_.push_back(x);
    }

    int pop() {
        if (empty()) { throw std::runtime_error("Stack kosong"); }
        int v = data_.back();
        data_.pop_back();
        return v;
    }
}

```

```

int peek() const {
    if (empty()) {
        throw std::runtime_error("Stack kosong");
    }
    return data_.back();
}

bool empty() const noexcept { return data_.empty(); }
int size() const noexcept { return static_cast<int>(data_.size()); }

private:
    int capacity_;
    std::vector<int> data_;
};

int main() {
    BoundedStack s{3};
    s.push(10);
    s.push(20);
    std::cout << "peek = " << s.peek() << "\n";
    std::cout << "pop  = " << s.pop() << "\n";
    std::cout << "size = " << s.size() << "\n";
    return 0;
}

```

RAII + Ownership

Ownership: siapa yang bertanggung jawab menutup resource? Objek yang memilikinya.

Destructor adalah konsekuensi responsibility.

```

// raii_file_logger.cpp
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>

class FileLogger {
public:
    explicit FileLogger(const std::string& filename) : file_(filename, std::ios::app) {
        if (!file_) { throw std::runtime_error("Gagal membuka file: " + filename); }
    }    // invariant: file_ valid terbuka selama objek hidup

    void log(const std::string& msg) {
        file_ << msg << "\n";
    }

    ~FileLogger() {}
        // Tidak perlu file_.close(); destructor ofstream akan menutup otomatis.
        // Ini RAII: resource dilepas saat objek mati.

private:
    std::ofstream file_;
};

```

```
int main() {  
    FileLogger logger("demo_log.txt");  
    logger.log("Hello from RAII");  
    logger.log("Objek logger akan menutup file saat keluar scope");  
    std::cout << "Selesai menulis log.\n";  
    return 0;  
}
```

const dan noexcept sebagai Kontrak API

```

// const_as_contract.cpp
#include <iostream>
#include <vector>
#include <stdexcept>

class Stack {
public:
    void push(int x) { data_.push_back(x); }

    int pop() {
        if (data_.empty()) throw std::runtime_error("empty");
        int v = data_.back();
        data_.pop_back();
        return v;
    }

    int peek() const { // Pembuat kelas menjamin peek tidak mengubah state objek
        if (data_.empty()) throw std::runtime_error("empty");
        return data_.back();
    }

    bool empty() const noexcept { return data_.empty(); } // Pembuat kelas menjamin pemeriksaan
                                                         // empty tidak gagal / exception

```

```

private:
    std::vector<int> data_;
};

// Kontrak: fungsi ini hanya membaca (observasi), tidak mengubah stack.
void print_top_if_any(const Stack& s) {
    if (!s.empty()) {
        std::cout << "top = " << s.peek() << "\n";
    } else {
        std::cout << "(empty)\n";
    }
}

int main() {
    Stack s;
    print_top_if_any(s);

    s.push(42);
    print_top_if_any(s);

    (void)s.pop();
    print_top_if_any(s);
    return 0;
}

```